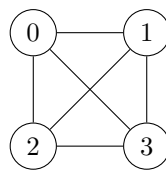


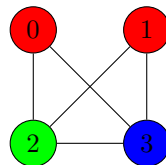
EXERCISE 1B*Last update 2025-10-23 (Version 038bbf0d)***3-Coloring**

Implement an algorithm which makes a graph 3-colorable by removing the least edges possible. A graph is 3-colorable if it is possible to assign one of 3 colors to each vertex, such that no two connected vertices share the same color.

For instance, consider the following graph:



This graph is not 3-colorable; however it can be made 3-colorable by removing the edge $(0, 1)$:



Thus $\{ (0, 1) \}$ is a minimal set of edges which must be removed to make this graph 3-colorable. Note that there are other such sets of equal length, for instance $\{ (0, 2) \}$, $\{ (0, 3) \}$ or $\{ (2, 3) \}$.

The problem of deciding whether a graph is 3-colorable is NP-complete and the same is true for the problem of finding a minimal set of edges to make a graph 3-colorable. Therefore an exact solution becomes infeasible for moderately large graph sizes. However, a randomized algorithm can be used to find a good approximation, i.e., to find a set of edges which is close to minimal.¹

A simple randomized algorithm for this problem generates a set of edges which must be removed to make a given graph 3-colorable by executing following steps:

- Generate a random (not necessarily valid) 3-coloring of the graph, i.e., randomly assign one of 3 colors to each vertex of the graph.
- Select all edges (u, v) for which the color of u is identical to the color of v . These edges need to be removed to make the graph 3-colorable.

Proof: Once these edges have been removed, the previously generated 3-coloring becomes valid. Thus removing these edges makes the graph 3-colorable.

¹You can read more about the complexity of the 3-colorability problem and about Monte Carlo randomized algorithms in general on Wikipedia:

https://en.wikipedia.org/wiki/Graph_coloring

https://en.wikipedia.org/wiki/Monte_Carlo_algorithm

These steps are executed repeatedly and the smallest set of edges so far is retained.

Although it might seem surprising given its simplicity, this algorithm has an expected runtime which is polynomial in the size of the graph and thus is on average much faster than an algorithm which tries to find an exact solution.

Instructions

In order to further reduce the runtime of this algorithm, multiple processes generate the random sets of edges in parallel and report their results to a supervisor process, which remembers the set with the least edges.

Write two programs: a **generator program** and a **supervisor program**. **Multiple generator processes** generate random solutions to the problem and report their solutions to **one supervisor process**. The supervisor process remembers the best solution so far. The processes communicate with each other by means of a circular buffer, which is implemented using shared semaphores and a shared memory.

Supervisor

The supervisor sets up the shared memory, the semaphores and initializes the circular buffer required for the communication with the generators. Then, it waits for the generators to write solutions to the circular buffer.

SYNOPSIS:

```
supervisor [-n limit] [-w delay]
```

The argument **limit** specifies a limit (integer value) for the number of generated solutions. If **limit** is omitted, it should be considered as infinite. The argument **delay** specifies a delay (in seconds) before reading the first solution from the buffer. If **delay** is omitted, it should be considered as zero.

Once initialization is complete, it waits for **delay** seconds², and then the supervisor reads the solutions from the circular buffer and remembers the best solution so far, i.e., the solution with the least edges. Every time a better solution than the previous best solution is found, the supervisor writes the new solution to *stderr*. If a generator writes a solution with 0 edges to the circular buffer, then the graph is 3-colorable and the supervisor terminates. In this case, the supervisor prints **The graph is 3-colorable!** to *stdout*.

Otherwise the supervisor keeps reading results from the circular buffer until it receives a **SIGINT**, a **SIGTERM** signal, or the number of read solutions from the circular buffer exceeds the value **limit**. In this case, the supervisor prints **The graph might not be 3-colorable, best solution removes N edges.** to *stdout* (where N is replaced with the number of edges of the best found solution).

Before terminating, the supervisor notifies all generators that they should terminate as well. This can be done by setting a variable in the shared memory, which is checked by the generator processes before writing to the buffer. The supervisor then unlinks all shared resources and exits.

²Only for this purpose, you may use the function `sleep()`. This delay is intended to give enough time to start multiple generators on the command line manually.

Generator

The generator program takes a graph as input. The program repeatedly generates a random solution to the problem as described on the first page and writes its result to the circular buffer. It repeats this procedure until it is notified by the supervisor to terminate.

The generator program takes as arguments the set of edges of the graph:

SYNOPSIS

```
generator EDGE1...
```

EXAMPLE

```
generator 0-1 0-2 0-3 1-2 1-3 2-3
```

Each positional argument is one edge; at least one edge must be given. An edge is specified by a string, with the indices of the two nodes it connects separated by a -. Note that the number of nodes of the graph is implicitly provided through the indices in the edges. The generator must not make any assumptions as to the number of edges that may be specified and programmatically accept a graph of any size (as well in the number edges as in the number of nodes)³. In the example above the generator program is called with the graph shown on the top of the first page.

The generator uses the algorithm described on the first page to generate random sets of edges which must be removed to make the given graph 3-colorable. It writes these edge sets to the circular buffer, one at a time; therefore a set of edges is a single element of the circular buffer. The generator may produce debug output, describing the edge sets which it writes to the circular buffer (only if the compile flag -DDEBUG is set).

³Practically, the number of edges which can be specified is limited by the system constant `ARG_MAX`. However, your program must not make any assumptions based on this (or other) limits.

Circular Buffer

The generators report their solutions to the supervisor by means of a circular buffer. The generators write their solutions to the write end of the circular buffer and the supervisor reads them from the read end.

A circular buffer is a common data structure which uses a single fixed-size buffer to implement a queue (i.e., a FIFO buffer). A circular buffer is essentially **an array of the elements** you want to pass through. Elements are written successively to the array and once the end of the array is reached, writing restarts from the beginning. Similarly, elements are also read successively from the array, restarting from the beginning upon reaching the end, thus reading the elements in the exact same order they have been written.⁴

Implement your circular buffer such that **reading and writing** to the buffer can happen **simultaneously**. Some synchronization is required in order to avoid overwriting data which has not been read yet and also to avoid trying to read from locations which have not been written yet. This is achieved using two semaphores: **one semaphore tracking the free space** in the circular buffer and **one semaphore tracking the used space**. The value of the free space semaphore corresponds to the amount of free space in the buffer array; it is initialized to the size of the buffer, since initially the circular buffer is empty. The value of the used space semaphore corresponds to the amount of used space in the buffer array; it is initialized to 0.

Upon **writing to the circular buffer**, the free space semaphore is decremented; if the buffer is currently full and there is no free space to write to, this intentionally blocks the write until space becomes available. After the write is complete, the used space semaphore is incremented, since the buffer now holds one more element which can be read.

Upon **reading from the circular buffer**, the used space semaphore is decremented; if the buffer is currently empty and there is no data to be read, this intentionally blocks the read until data becomes available. After the read is complete, the free space semaphore is incremented, since the position occupied by the element which has been read just has been freed up.

Since multiple generators write to the circular buffer, an additional semaphore will be required to guarantee a **mutually exclusive access to the write end** of the circular buffer.

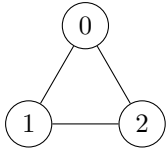
Hints

- Store the graph in a way which is suitable for the implementation of the randomized algorithm.
- Think of a suitable structure to store a set of edges in the circular buffer. Choose a limit on the maximum number of edges which it can contain. Since we are looking for small solutions, the generator can discard any solutions which contain too many edges instead of writing them to the circular buffer. You may choose a limit as low as 8 edges.
- Select a reasonable size for the circular buffer. If the size is too small, then the generators will spend a lot of time waiting for free space to become available. If the size is too large, the circular buffer is wasting resources. The size of the shared memory should not exceed 4 KiB.

⁴Visualizations of circular buffers can be found on Wikipedia:
https://en.wikipedia.org/wiki/Circular_buffer

Examples

Simple 3-colorable graph



This graph is already 3-colorable.

Invocation of the supervisor:

```
1 $ ./supervisor
2 Solution with 1 edges: 0-2
3 The graph is 3-colorable!
```

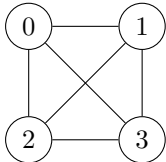
Invocation of one generator:

```
1 $ ./generator 0-1 0-2 1-2
```

Invocation of 10 generators in parallel:

```
1 $ for i in {1..10}; do (./generator 0-1 0-2 1-2 &); done
```

Graph from the first page



Removing one edge is sufficient to make this graph 3-colorable, as seen on the first page.

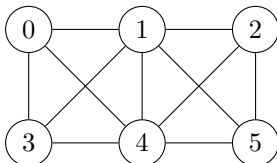
Invocation of the supervisor:

```
1 $ ./supervisor -n 1000
2 Solution with 2 edges: 0-3 1-2
3 Solution with 1 edges: 0-1
4 The graph might not be 3-colorable,
5 best solution removes 1 edges.
```

Invocation of the generator:

```
1 $ ./generator 0-1 0-2 0-3 1-2 1-3 2-3
```

More complex graph



This graph can again be made 3-colorable by removing a single edge.

Invocation of the supervisor:

```
1 $ ./supervisor -n 1000
2 Solution with 4 edges: 0-3 0-4 1-2 3-4
3 Solution with 3 edges: 0-4 1-2 4-5
4 Solution with 2 edges: 1-3 1-5
5 Solution with 1 edges: 1-4
6 The graph might not be 3-colorable,
7 best solution removes 1 edges.
```

Invocation of the generator:

```
1 $ ./generator 0-1 0-3 0-4 1-2 1-3 1-4 1-5 2-4 2-5 3-4 4-5
```

If you want to test your implementation with a challenging graph, try this one ⁵:

0-1 0-2 0-3 1-28 1-29 2-30 2-31 3-32 3-33 4-6 4-14 4-16 5-7 5-15 5-17 6-7 6-18 7-19
8-9 8-12 8-23 9-13 9-22 10-15 10-19 10-25 11-14 11-18 11-24 12-17 12-27 13-16 13-26
14-23 15-22 16-21 17-20 18-21 19-20 20-31 21-30 22-33 23-32 24-27 24-29 25-26 25-29
26-28 27-28 30-33 31-32

Note: This graph is 3-colorable, therefore one of your generators should eventually come up with a solution with 0 edges.

⁵Source: Graph #3349 from the House of Graphs: <https://hog.grinvin.org/ViewGraphInfo.action?id=3349>

Testcases

Input is shown in blue color. Output to *stdout* (and *stderr*) is shown in black. (Note that in the following output sections `EXIT_SUCCESS` equals 0, and `EXIT_FAILURE` equals 1. Refer to `stdlib.h` for further details.) `^C` indicates `CTRL+C`, `^D` indicates `CTRL+D`. The placeholder `<usage message>` must be replaced by a proper usage message (printed to *stdout*), `<error message>` must be replaced by a meaningful error message (which is printed to *stderr*).

Testcase 01: simple-1

Start your supervisor first.

```
1 $ ./supervisor
2 The graph is 3-colorable!
3 $ echo $?
4 0
```

Then start the generator.

```
1 $ ./generator 0-1 0-2 1-2
2 $ echo $?
3 0
```

Testcase 02: simple-2

Start your supervisor first.

```
1 $ ./supervisor -n 500
2 The graph might not be 3-colorable, best \
   solution removes 1 edges.
3 $ echo $?
4 0
```

Then start the generator.

```
1 $ ./generator 0-1 0-2 0-3 1-2 1-3 2-3
2 $ echo $?
3 0
```

Testcase 03: multi-1

Start your supervisor first.

```
1 $ ./supervisor -n 100000 -w 2
2 The graph might not be 3-colorable, best \
   solution removes 6 edges.
3 $ echo $?
4 0
```

Then start the generators.

```
1 $ for i in `seq 0 2`; do ( ./generator 0-1 0-2 \
   0-3 0-4 0-5 0-6 0-7 1-2 1-3 1-4 1-5 1-6 1-7 \
   2-3 2-4 2-5 2-6 2-7 3-4 3-5 3-6 3-7 4-5 4-6 \
   4-7 5-6 6-7 ; echo "generator returned $?" \
   ) & done;
2 generator returned 0
3 generator returned 0
4 generator returned 0
```

Testcase 04: multi-2-petersen

Start your supervisor first.

```
1 $ ./supervisor -n 100000 -w 2
2 The graph is 3-colorable!
3 $ echo $?
4 0
```

Then start the generators.

```
1 $ for i in `seq 0 2`; do ( ./generator 0-1 0-2 \
   0-3 1-4 1-5 2-6 2-9 3-7 3-8 4-6 4-8 5-7 5-9 \
   6-7 8-9 ; echo "generator returned $?" ) & \
   done;
2 generator returned 0
3 generator returned 0
4 generator returned 0
```

Testcase 05: multi-3-sousselier

Start your supervisor first.

```
1 $ ./supervisor -n 1000000 -w 2
2 The graph is 3-colorable!
3 $ echo $?
4 0
```

Then start the generators.

```
1 $ for i in `seq 0 2`; do ( ./generator 0-8 0-9 \
    0-15 1-7 1-10 1-15 2-5 2-13 2-15 3-6 3-14 \
    3-15 4-11 4-12 4-15 5-6 5-12 6-11 7-8 7-13 \
    8-14 9-12 9-13 10-11 10-14 11-13 12-14 ; \
    echo "generator returned $?" ) & done;
2 generator returned 0
3 generator returned 0
4 generator returned 0
```

Testcase 06: multi-4-tesseract

Start your supervisor first.

```
1 $ ./supervisor -n 1000000 -w 2
2 The graph is 3-colorable!
3 $ echo $?
4 0
```

Then start the generators.

```
1 $ for i in `seq 0 2`; do ( ./generator 0-1 0-2 \
    1-13 1-14 1-16 2-11 2-12 2-15 3-4 3-5 3-9 \
    3-10 4-6 4-7 4-8 5-6 5-11 5-12 6-13 6-14 \
    7-9 7-13 7-16 8-10 8-14 8-16 9-12 9-15 \
    10-11 10-15 11-14 12-13 15-16 ; echo \
    "generator returned $?" ) & done;
2 generator returned 0
3 generator returned 0
4 generator returned 0
```

Testcase 07: multi-5-clebsch

Start your supervisor first.

```
1 $ ./supervisor -n 1000000 -w 2
2 The graph might not be 3-colorable, best \
    solution removes 3 edges.
3 $ echo $?
4 0
```

Then start the generators.

```
1 $ for i in `seq 0 4`; do ( ./generator 0-1 0-2 \
    0-3 0-4 0-5 1-6 1-7 1-8 1-9 2-6 2-10 2-11 \
    2-12 3-7 3-10 3-13 3-14 4-8 4-11 4-13 4-15 \
    5-9 5-12 5-14 5-15 6-13 6-14 6-15 7-11 7-12 \
    7-15 8-10 8-12 8-14 9-10 9-11 9-13 10-15 \
    11-14 12-13 ; echo "generator returned $?" \
    ) & done;
2 generator returned 0
3 generator returned 0
4 generator returned 0
5 generator returned 0
6 generator returned 0
```

Testcase 10: no-supervisor

```
1 $ ./generator 0-1 0-2 1-2
2 <error message>
3 $ echo $?
4 1
```

Testcase 11: usage

```
1 $ ./supervisor -x
2 <usage message>
3 $ echo $?
4 1
```

Recommendations and Guidelines

In order to make all solutions more uniform and to avoid problems with different C dialects, please try to follow the following guidelines.

General Guidelines

Try to follow the following guidelines closely.

1. All source files of your program(s) should compile with

```
$ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
-D_POSIX_C_SOURCE=200809L -g -c filename.c
```

without *warnings and info messages* and your program(s) should link without warnings too.
2. There should be a Makefile implementing the targets: **all** to build the program(s) (i.e. generate executables) from the sources (this must be the first target in the Makefile); **clean** to delete all files that can be built from your sources with the Makefile.
3. Make all targets of your Makefile idempotent, i.e. execution of **make clean**; **make clean** should yield the same result as **make clean**, and should not fail with an error.
4. The program shall operate according to the specification/assignment without major issues (e.g., segmentation fault, memory corruption).
5. Arguments should be parsed according to UNIX conventions (we strongly encourage the use of **getopt(3)**). The program should conform to the given synopsis/usage in the assignment. If the synopsis is violated (e.g., unspecified options or too many arguments), the program should terminate with the usage message containing the program name and the correct calling syntax. Argument handling should also be implemented for programs without arguments.
6. Correct (=normal) termination should include a cleanup of resources.
7. Upon success the program shall terminate with exit code 0, in case of errors with an exit code greater than 0. We recommend to use the macros **EXIT_SUCCESS** and **EXIT_FAILURE** (defined in **stdlib.h**) to enable portability of the program.
8. If a function indicates an error with its return value, it *should* be checked in general. If the subsequent code depends on the successful execution of a function (e.g., resource allocation), it is very *important* to check the return value.
9. Functions that do not take any parameters should be declared with **void** in the signature, e.g., **int get_random_int(void);**.
10. Procedures (i.e., functions that do not return a value) should be declared as **void**.
11. Error messages shall be written to **stderr** and should contain the program name **argv[0]**.
12. Do not use the following functions to avoid crashes due to invalid inputs:
 - **gets(3)**; use **fgets(3)** instead.
 - **scanf(3)** and **fscanf(3)**; use **fgets(3)** and **sscanf(3)** instead.
 - **atoi(3)** and **atol(3)**; use **strtol(3)** instead.
13. Documentation is a nice bonus. Format the documentation in Doxygen style (see Wiki and Doxygen's intro).

14. Write meaningful comments. For example, meaningful comments describe the algorithm, or why a particular solution has been chosen, if there seems to be an easier solution at a first glance. Avoid comments that just repeat the code itself
(e.g., `i = i + 1; /* i is incremented by one */`).
15. The documentation of a module should include: name of the module, name and student id of the author (`@author` tag), purpose of the module (`@brief`, `@details` tags) and creation date of the module (`@date` tag).
The Makefile should also include a header, with author and program name at least.
16. Each function shall be documented either before the declaration or the implementation. It should include purpose (`@brief`, `@details` tags), description of parameters and return value (`@param`, `@return` tags) and description of global variables the function uses (`@details` tag).
You should also document `static` functions (see `EXTRACT_STATIC` in the file `Doxyfile`). Document visible/exported functions in the header file and local (`static`) functions in the C file. Document variables, constants and types (especially `structs`) too.
17. Documentation, names of variables and constants is best done in English.
18. Internal functions shall be marked with the `static` qualifier and are not allowed to be exported (e.g., in a header file). Only functions that are used by other modules shall be declared in the header file.
19. All exercises shall be solved with functions of the C standard library. If a required function is not available in the standard library, you can use other (external) functions too. Avoid reinventing the wheel (e.g., re-implementation of `strcmp`).
20. Name of constants shall be written in upper case, names of variables in lower case (possibly with the first letter capitalized).
21. Use meaningful variable and constant names (e.g., also semaphores and shared memories).
22. Avoid using global variables as far as possible.
23. All boundaries shall be defined as constants (macros). Avoid arbitrary boundaries. If boundaries are necessary, treat its crossing.
24. Avoid side effects with `&&` and `||`, e.g., write `if(b != 0) c = a/b;` instead of `if(b != 0 && c = a/b)`.
25. Each `switch` block should contain a `default` case. If the case is not reachable, write `assert(0)` to this case (defensive programming).
26. Logical values shall be treated with logical operators, numerical values with arithmetic operators (e.g., test 2 strings for equality by `strcmp(...) == 0` instead of `!strcmp(...)`).
27. Indent your source code consistently (there are tools for that purpose, e.g., `indent`).
28. Avoid tricky arithmetic statements. Programs are written once, but read many times. Your program is not always better if it is shorter!
29. For all I/O operations (read/write from/to `stdin`, `stdout`, files, sockets, pipes, etc.) use *either* standard I/O functions (`fdopen(3)`, `fopen(3)`, `fgets(3)`, etc.) *or* POSIX functions (`open(2)`, `read(2)`, `write(2)`, etc.). Remember, standard I/O functions are buffered. Mixing standard I/O functions and POSIX functions to access a common file descriptor can lead to undefined behaviour and is therefore a bad idea.
30. If asked in the assignment, you should implement signal handling (`SIGINT`, `SIGTERM`). You must use only *async-signal-safe* functions in your signal handlers.
31. Close files, free dynamically allocated memory, and remove resources after usage.

32. Don't waste resources due to inconvenient programming. Header files shall not include implementation parts (exception: macros).
33. To comply with the given testcases, the program output must exactly match the given specification. Therefore you should only print debug information if the compile flag `-DDEBUG` is set, in order to avoid problems with the automatic checker.

Exercise 1B Guidelines

The following points are import for exercise 1b.

1. Correct use of named semaphores (`sem_open(3)`, `sem_close(3)` `sem_unlink(3)`) and POSIX shared memory (`shm_overview(7)`) for inter-process communication of separated programs (e.g., server and client).
Use your matriculation number as prefix in the names of all resources, to avoid problems when testing. (You don't want to use the same ressources as another student!)
2. "Busy waiting" is a very bad solution. Don't do it. (Busy waiting is the repeated check of a condition in a loop for synchronization purposes.)
3. Synchronization with `sleep` is also a very bad idea. Use the proper tools for synchronization.